

Neural Network Example

Kachun Huang

2025-03-03

Neural Network Classification Example

Import Libraries

```
library(neuralnet)
```

```
## Warning: package 'neuralnet' was built under R version 4.3.3
```

```
library(MASS)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr    1.5.1
## v ggplot2     3.5.1      v tibble     3.2.1
## v lubridate  1.9.3      v tidyr      1.3.0
## v purrr       1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::compute() masks neuralnet::compute()
## x dplyr::filter()  masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## x dplyr::select()  masks MASS::select()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(leaps)
```

```
## Warning: package 'leaps' was built under R version 4.3.3
```

```
library(car)
```

```
## Warning: package 'car' was built under R version 4.3.3
```

```
## Loading required package: carData
##
## Attaching package: 'car'
##
```

```
## The following object is masked from 'package:dplyr':  
##  
##      recode  
##  
## The following object is masked from 'package:purrr':  
##  
##      some
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.3.3
```

```
## Loading required package: lattice  
##  
## Attaching package: 'caret'  
##  
## The following object is masked from 'package:purrr':  
##  
##      lift
```

```
library(dplyr)  
library(lmtest)
```

```
## Warning: package 'lmtest' was built under R version 4.3.3
```

```
## Loading required package: zoo
```

```
## Warning: package 'zoo' was built under R version 4.3.3
```

```
##  
## Attaching package: 'zoo'  
##  
## The following objects are masked from 'package:base':  
##  
##      as.Date, as.Date.numeric
```

Import Dataset

The banknote.csv data (see attached) were extracted from images that were taken from genuine and forged banknote-like specimens. Yes, this is a Catch Me if You Can story. For digitization, an industrial camera usually used for print inspection was used. The final images have 400 x 400 pixels. (*3D would be called the voxels) Wavelet Transform tool were used to extract features from images.

There are 1,372 banknotes, and 5 variables:

1. variance of Wavelet Transformed image (continuous)
2. skewness of Wavelet Transformed image (continuous)
3. curtosis of Wavelet Transformed image (continuous)

4. entropy of image (continuous)
5. class (binary) – this is the response variable of interest, 1 (forged) or 0 (genuine)

```
bank <- read.table("/Users/thomashuang/Downloads/banknote.csv",sep=",",header = TRUE)
head(bank)
```

```
##   variance skewness curtosis  entropy class
## 1  3.62160   8.6661  -2.8073 -0.44699     1
## 2  4.54590   8.1674  -2.4586 -1.46210     1
## 3  3.86600  -2.6383   1.9242  0.10645     1
## 4  3.45660   9.5228  -4.0112 -3.59440     1
## 5  0.32924  -4.4552   4.5718 -0.98880     1
## 6  4.36840   9.6718  -3.9606 -3.16250     1
```

```
str(bank)
```

```
## 'data.frame':   1372 obs. of  5 variables:
## $ variance: num  3.622 4.546 3.866 3.457 0.329 ...
## $ skewness: num  8.67 8.17 -2.64 9.52 -4.46 ...
## $ curtosis: num -2.81 -2.46 1.92 -4.01 4.57 ...
## $ entropy : num -0.447 -1.462 0.106 -3.594 -0.989 ...
## $ class : int  1 1 1 1 1 1 1 1 1 1 ...
```

Data Pre-Processing

Remove Missing Values

```
bank <- na.omit(bank)
str(bank)
```

```
## 'data.frame':   1372 obs. of  5 variables:
## $ variance: num  3.622 4.546 3.866 3.457 0.329 ...
## $ skewness: num  8.67 8.17 -2.64 9.52 -4.46 ...
## $ curtosis: num -2.81 -2.46 1.92 -4.01 4.57 ...
## $ entropy : num -0.447 -1.462 0.106 -3.594 -0.989 ...
## $ class : int  1 1 1 1 1 1 1 1 1 1 ...
```

None of the observation have missing values.

Factorize response

Data Splitting

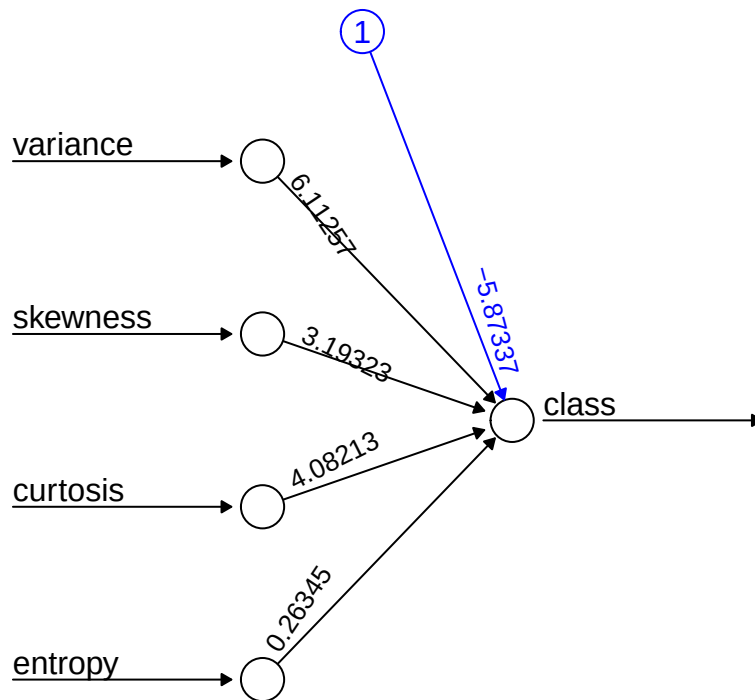
```
set.seed(123)
training.samples <- bank$class %>% createDataPartition(p=0.75, list=FALSE)
train.data <- bank[training.samples,]
test.data <- bank[-training.samples,]
```

Perceptron Model with No Hidden Layers

with SSE Loss function

```
p_model_sse <- neuralnet(  
  class ~.,  
  train.data,  
  hidden = 0, # no hidden layer  
  err.fct = "sse", # sse loss function  
  linear.output = FALSE, # non-linear output for classification  
  act.fct = "logistic"# logistic activation function  
)
```

```
plot(p_model_sse, rep="best")
```



Plot model

Error: 3.147732 Steps: 2702

```
pred_sse <- predict(p_model_sse, test.data)  
pred_sse_class <- ifelse(pred_sse > 0.5, 1, 0)
```

Predict on Test Data

```
cm1 <- confusionMatrix(factor(pred_sse_class), factor(test.data$class), positive = "1")
cm1
```

Confusion Matrix

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0 160    1
##           1   1 181
##
##           Accuracy : 0.9942
##           95% CI : (0.9791, 0.9993)
##           No Information Rate : 0.5306
##           P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.9883
##
##           Mcnemar's Test P-Value : 1
##
##           Sensitivity : 0.9945
##           Specificity : 0.9938
##           Pos Pred Value : 0.9945
##           Neg Pred Value : 0.9938
##           Prevalence : 0.5306
##           Detection Rate : 0.5277
##           Detection Prevalence : 0.5306
##           Balanced Accuracy : 0.9941
##
##           'Positive' Class : 1
##
```

When positive class = “1” Detect forged

Actual/Predicted	Negative (Actual 0)	Positive (Actual 1)
Negative (Predicted 0)	**True Negative (TN)**	**False Negative(FN)**
Positive (Predicted 1)	**False Positive(FP)**	**True Positive(TP)**

Sensitivity $Sensitivity = P(\text{true positive}|\text{correct}) = \frac{TP}{TP+FN}$

```
181/(181+1)
```

```
## [1] 0.9945055
```

```
cm1$byClass["Sensitivity"]
```

```
## Sensitivity
## 0.9945055
```

Specificity $Specificity = P(\text{true negative}|\text{correct}) = \frac{TN}{TN+FP}$

```
160 / (160+1)
```

```
## [1] 0.9937888
```

```
cm1$byClass["Specificity"]
```

```
## Specificity  
## 0.9937888
```

Overall Accuracy $P(\text{Predicted Correctly}) = \frac{TP+TN}{TP+TN+FP+FN}$

```
(181+160)/(181+160+1+1)
```

```
## [1] 0.9941691
```

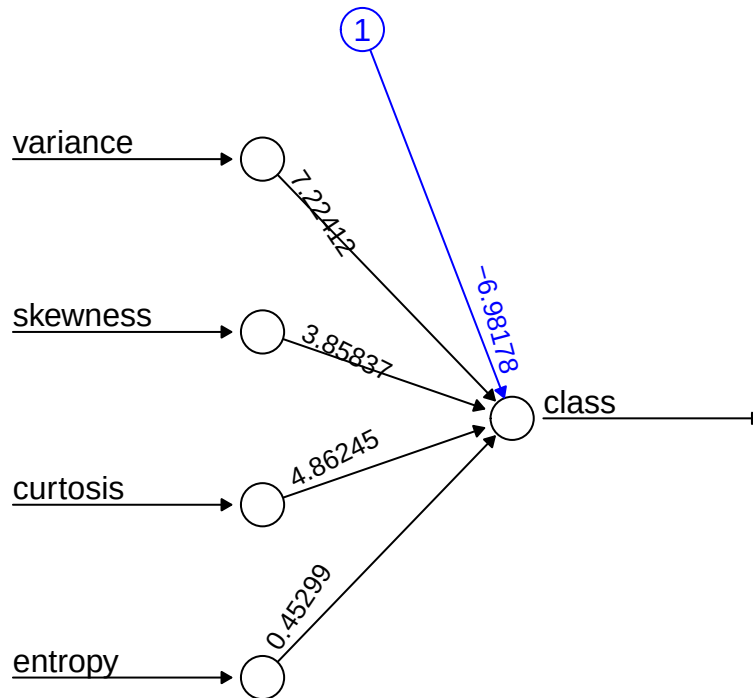
```
cm1$overall["Accuracy"]
```

```
## Accuracy  
## 0.9941691
```

with CE Loss function

```
p_model_ce <- neuralnet(  
  class ~.,  
  train.data,  
  hidden = 0,  
  err.fct = "ce",  
  linear.output = FALSE,  
  act.fct = "logistic"  
)
```

```
plot(p_model_ce, rep="best")
```



Plot model

Error: 19.689505 Steps: 2749

```

pred_ce <- predict(p_model_ce, test.data)
pred_ce_class <- ifelse(pred_ce > 0.5, 1, 0)

```

Predict on Test Data

```

cm2 <- confusionMatrix(factor(pred_ce_class), factor(test.data$class), positive = "1")
cm2

```

Confusion Matrix

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0 160    3
##           1   1 179
##
##           Accuracy : 0.9883
##           95% CI : (0.9704, 0.9968)
##           No Information Rate : 0.5306
##           P-Value [Acc > NIR] : <2e-16
##

```

```
##                Kappa : 0.9766
##
##  McNemar's Test P-Value : 0.6171
##
##          Sensitivity : 0.9835
##          Specificity : 0.9938
##          Pos Pred Value : 0.9944
##          Neg Pred Value : 0.9816
##          Prevalence : 0.5306
##          Detection Rate : 0.5219
##          Detection Prevalence : 0.5248
##          Balanced Accuracy : 0.9887
##
##          'Positive' Class : 1
##
```

When positive class = “1” Detect forged

Actual/Predicted	Negative (Actual 0)	Positive (Actual 1)
Negative (Predicted 0)	**True Negative (TN)**	**False Negative(FN)**
Positive (Predicted 1)	**False Positive(FP)**	**True Positive(TP)**

Sensitivity $Sensitivity = P(\text{true positive}|\text{correct}) = \frac{TP}{TP+FN}$

```
179 / (179+3)
```

```
## [1] 0.9835165
```

```
cm2$byClass["Sensitivity"]
```

```
## Sensitivity
##    0.9835165
```

Specificity $Specificity = P(\text{true negative}|\text{correct}) = \frac{TN}{TN+FP}$

```
160/(160+1)
```

```
## [1] 0.9937888
```

```
cm2$byClass["Specificity"]
```

```
## Specificity
##    0.9937888
```

Overall Accuracy $P(\text{Predicted Correctly}) = \frac{TP+TN}{TP+TN+FP+FN}$

```
(160+178)/(160+178+1+3)
```

```
## [1] 0.9883041
```

```
cm2$overall["Accuracy"]
```

```
## Accuracy  
## 0.9883382
```

Logistic Regression

Fit model

```
logit_model <- glm(class ~ variance + skewness + curtosis + entropy,  
  data = train.data,  
  family = "binomial")
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
summary(logit_model)
```

```
##  
## Call:  
## glm(formula = class ~ variance + skewness + curtosis + entropy,  
##      family = "binomial", data = train.data)  
##  
## Coefficients:  
##              Estimate Std. Error z value Pr(>|z|)  
## (Intercept)  -7.0385      1.6445  -4.280 1.87e-05 ***  
## variance      7.2886      1.7688   4.121 3.78e-05 ***  
## skewness      3.8913      0.9077   4.287 1.81e-05 ***  
## curtosis      4.9051      1.1666   4.205 2.62e-05 ***  
## entropy       0.4592      0.3561   1.289  0.197  
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
##  
## (Dispersion parameter for binomial family taken to be 1)  
##  
##      Null deviance: 1409.774  on 1028  degrees of freedom  
## Residual deviance:   39.378  on 1024  degrees of freedom  
## AIC: 49.378  
##  
## Number of Fisher Scoring iterations: 12
```

```
pred_logit <- predict(logit_model, test.data)  
pred_logit_class <- ifelse(pred_logit > 0.5, 1,0)
```

Predict on Test Data

```
cm3 <- confusionMatrix(factor(pred_logit_class),factor(test.data$class), positive = "1")
cm3
```

Confusion Matrix

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0 160    3
##           1   1 179
##
##           Accuracy : 0.9883
##           95% CI : (0.9704, 0.9968)
##           No Information Rate : 0.5306
##           P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.9766
##
## Mcnemar's Test P-Value : 0.6171
##
##           Sensitivity : 0.9835
##           Specificity : 0.9938
##           Pos Pred Value : 0.9944
##           Neg Pred Value : 0.9816
##           Prevalence : 0.5306
##           Detection Rate : 0.5219
##           Detection Prevalence : 0.5248
##           Balanced Accuracy : 0.9887
##
##           'Positive' Class : 1
##
```

When positive class = “1” Detect forged

Actual/Predicted	Negative (Actual 0)	Positive (Actual 1)
Negative (Predicted 0)	**True Negative (TN)**	**False Negative(FN)**
Positive (Predicted 1)	**False Positive(FP)**	**True Positive(TP)**

Sensitivity $Sensitivity = P(\text{true positive}|\text{correct}) = \frac{TP}{TP+FN}$

```
179 / (3+179)
```

```
## [1] 0.9835165
```

```
cm3$byClass["Sensitivity"]
```

```
## <NA>
## NA
```

Specificity $Specificity = P(\text{true negative}|\text{correct}) = \frac{TN}{TN+FP}$

```
160/(160+1)
```

```
## [1] 0.9937888
```

```
cm3$byClass["Specificity"]
```

```
## Specificity  
## 0.9937888
```

Overall Accuracy $P(\text{Predicted Correctly}) = \frac{TP+TN}{TP+TN+FP+FN}$

```
(179+160)/(179+160+1+3)
```

```
## [1] 0.9883382
```

```
cm3$overall["Accuracy"]
```

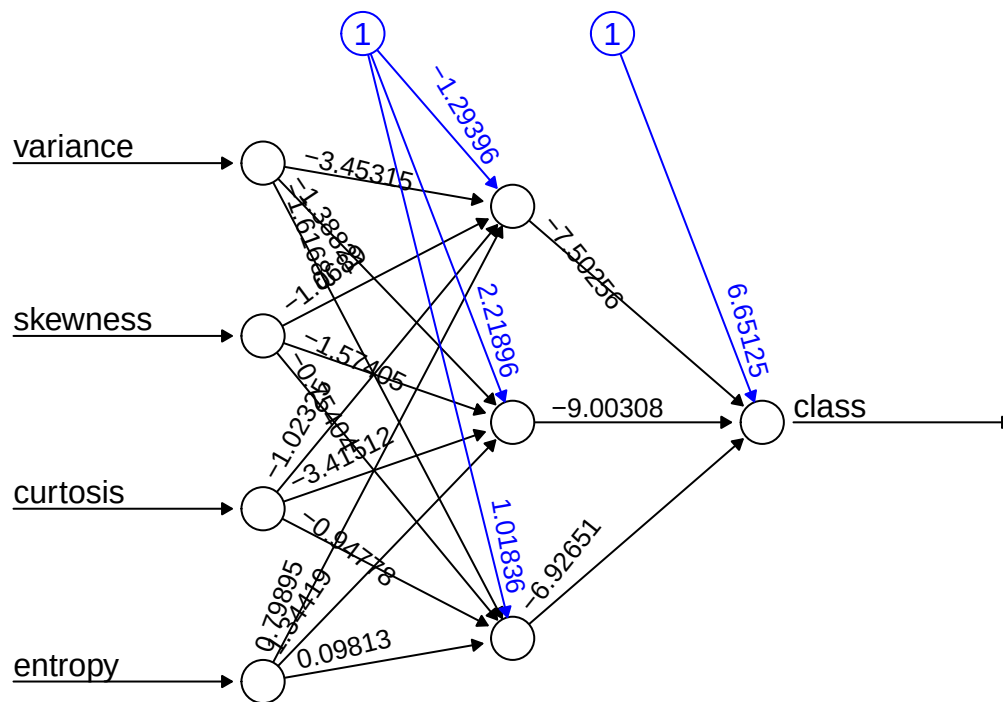
```
## Accuracy  
## 0.9883382
```

Perceptron Model with 1 Hidden Layers and 3 neurons

with SSE Loss function

```
p_model_layer_sse <- neuralnet(  
  class ~.,  
  train.data,  
  hidden = 3,  
  err.fct = "sse",  
  linear.output = FALSE,  
  act.fct = "logistic"  
)
```

```
plot(p_model_layer_sse, rep="best")
```



Plot model

Error: 0.00783 Steps: 86

```
pred_layer_sse <- predict(p_model_layer_sse, test.data)
pred_layer_sse_class <- ifelse(pred_layer_sse > 0.5,1,0)
```

Predict on Test Data

```
cm4 <- confusionMatrix(factor(pred_layer_sse_class), factor(test.data$class), positive = "1")
cm4
```

Confusion Matrix

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0 161    0
##           1   0 182
##
##           Accuracy : 1
##           95% CI : (0.9893, 1)
##           No Information Rate : 0.5306
##           P-Value [Acc > NIR] : < 2.2e-16
##
```

```
##                Kappa : 1
##
##  McNemar's Test P-Value : NA
##
##          Sensitivity : 1.0000
##          Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 1.0000
##          Prevalence : 0.5306
##          Detection Rate : 0.5306
##          Detection Prevalence : 0.5306
##          Balanced Accuracy : 1.0000
##
##          'Positive' Class : 1
##
```

When positive class = “1” Detect forged

	Actual/Predicted	Negative (Actual 0)	Positive (Actual 1)
Negative (Predicted 0)			
Positive (Predicted 1)			

True Negative (TN) | **False Negative (FN)** |
False Positive (FP) | **True Positive (TP)** |

Sensitivity $Sensitivity = P(\text{true positive} | \text{correct}) = \frac{TP}{TP+FN}$

```
182/(182+0)
```

```
## [1] 1
```

```
cm4$byClass["Sensitivity"]
```

```
## Sensitivity
##          1
```

Specificity $Specificity = P(\text{true negative} | \text{correct}) = \frac{TN}{TN+FP}$

```
161 / (0+161)
```

```
## [1] 1
```

```
cm4$byClass["Specificity"]
```

```
## Specificity
##          1
```

Overall Accuracy $P(\text{Predicted Correctly}) = \frac{TP+TN}{TP+TN+FP+FN}$

```
(161+182)/(161+182+0+0)
```

```
## [1] 1
```

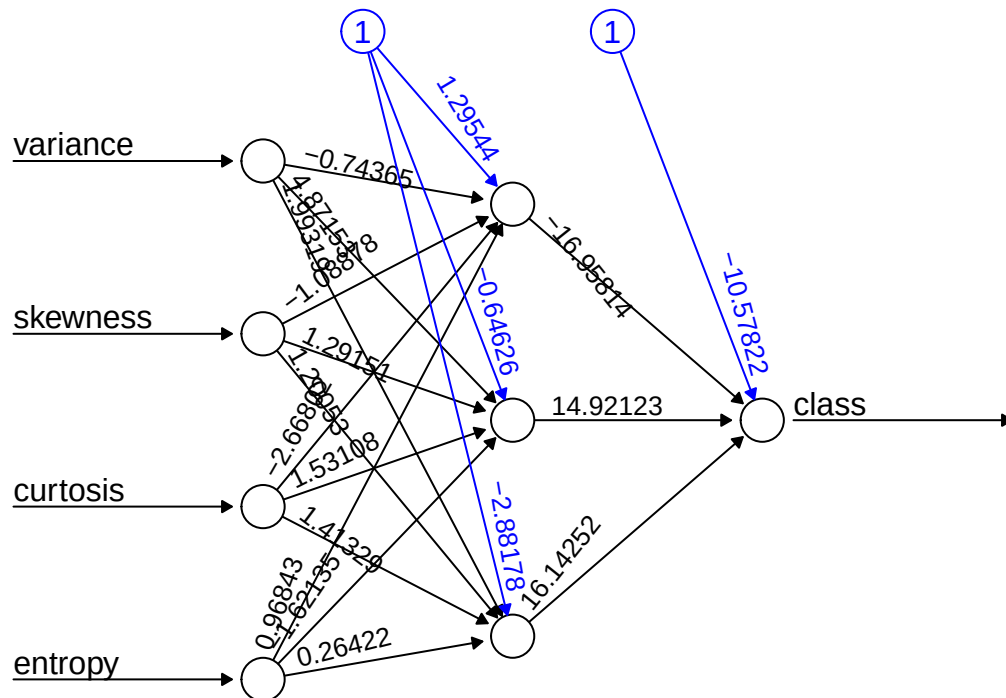
```
cm4$overall["Accuracy"]
```

```
## Accuracy  
##      1
```

with CE Loss function

```
p_model_layer_ce <- neuralnet(  
  class ~.,  
  train.data,  
  hidden = 3,  
  err.fct = "ce",  
  linear.output = FALSE,  
  act.fct = "logistic"  
)
```

```
plot(p_model_layer_ce, rep="best")
```



Plot model

Error: 0.02425 Steps: 174

```
pred_layer_ce <- predict(p_model_layer_ce, test.data)  
pred_layer_ce_class <- ifelse(pred_layer_ce > 0.5, 1, 0)
```

Predict on Test Data

```
cm5 <- confusionMatrix(factor(pred_layer_ce_class), factor(test.data$class), positive = "1")
cm5
```

Confusion Matrix

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0 161    0
##           1   0 182
##
##           Accuracy : 1
##           95% CI : (0.9893, 1)
##           No Information Rate : 0.5306
##           P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 1
##
## Mcnemar's Test P-Value : NA
##
##           Sensitivity : 1.0000
##           Specificity : 1.0000
##           Pos Pred Value : 1.0000
##           Neg Pred Value : 1.0000
##           Prevalence : 0.5306
##           Detection Rate : 0.5306
##           Detection Prevalence : 0.5306
##           Balanced Accuracy : 1.0000
##
##           'Positive' Class : 1
##
```

When positive class = “1” Detect forged

Actual/Predicted	Negative (Actual 0)	Positive (Actual 1)
Negative (Predicted 0)	**True Negative (TN)**	**False Negative(FN)**
Positive (Predicted 1)	**False Positive(FP)**	**True Positive(TP)**

Sensitivity $Sensitivity = P(\text{true positive}|\text{correct}) = \frac{TP}{TP+FN}$

```
182/(182+0)
```

```
## [1] 1
```

```
cm5$byClass["Sensitivity"]
```

```
## Sensitivity
##           1
```

Specificity $Specificity = P(\text{true negative}|\text{correct}) = \frac{TN}{TN+FP}$

```
(161)/(161+0)
```

```
## [1] 1
```

```
cm5$byClass["Specificity"]
```

```
## Specificity  
##          1
```

Overall Accuracy $P(\text{Predicted Correctly}) = \frac{TP+TN}{TP+TN+FP+FN}$

```
(161+182)/(161+182+0+0)
```

```
## [1] 1
```

```
cm5$overall["Accuracy"]
```

```
## Accuracy  
##          1
```

Result

The prediction with hidden layer is better than no hidden layer in this task.